

Entwicklungsprojekt

Asia Restaurant "All you can Eat" - Sensor-Dashboard

Projekttitlel	Intelligentes Sensor-Dashboard für ein Asia-Restaurant zur Auslastungsüberwachung mit linearer Regressionsanalyse
Projektteam	Normen, Aslan, Emilio
Github	https://github.com/AslanX0/pi_project_vre/tree/main
Zeitraum	Oktober 2025 – Februar 2026
Hardware	Raspberry Pi 4 Model B, BME680, RCWL-0516
Abgabedatum	27. Februar 2026

1. Projektbeschreibung

1.1 Zielsetzung

Ziel des Projekts ist die Entwicklung eines intelligenten Sensor-Dashboards für ein Asia-Restaurant. Das System erfasst kontinuierlich Umgebungsdaten (Temperatur, Luftfeuchtigkeit, Luftdruck, VOC-Gaswiderstand, Bewegung) über einen Raspberry Pi und stellt diese in Echtzeit auf einer Weboberfläche dar.

Zusätzlich wird mithilfe einer **linearen Regressionsanalyse** der Zusammenhang zwischen der geschätzten Personenanzahl im Restaurant und der Raumtemperatur modelliert, um Vorhersagen für verschiedene Auslastungsszenarien zu ermöglichen.

1.2 Eingesetzte Hardware

Komponente	Beschreibung	Funktion
Raspberry Pi 4 Model B	Einplatinencomputer (ARM, 4 GB RAM)	Steuereinheit, Webserver, Datenbank-Host
BME680	Umgebungssensor (Bosch)	Temperatur, Luftfeuchtigkeit, Luftdruck, VOC-Gaswiderstand
RCWL-0516	Mikrowellen-Bewegungssensor	Erkennung von Personenbewegungen im Raum

1.3 Eingesetzte Software & Technologien

Technologie	Einsatzbereich
Python 3	Gesamte Backend-Logik, Sensorauslesung, Datenverarbeitung
FastAPI	REST-API-Server, stellt alle Daten als JSON-Endpunkte bereit
MariaDB / MySQL	Persistente Speicherung der Sensordaten
HTML / CSS / JS	Einseitige Webanwendung (SPA) mit Chart.js für Diagramme
NumPy	Berechnung der linearen Regression (OLS-Methode)
RPi.GPIO / bme680	Hardwareanbindung auf dem Raspberry Pi

1.4 Systemarchitektur

Das System gliedert sich in drei Schichten:

- Sensorschicht:** BME680 + RCWL-0516 werden alle 300 Sekunden ausgelesen (main.py). Die Daten werden direkt in die MariaDB-Datenbank geschrieben.

- **Backend-Schicht:** FastAPI (app.py) stellt REST-API-Endpunkte bereit. Hintergrundprozesse übernehmen automatisch Personenschätzung, Regressionsmodell-Training und Datenlöschung nach 30 Tagen.
- **Frontend-Schicht:** Einseitige Webanwendung mit vier Tabs – Dashboard, Regression, Sensoren, Verlauf. Wird alle 30 Sekunden automatisch aktualisiert.

1.5 Projektstruktur

```

pipproject/
├── .gitignore
├──
├── backend/
│   ├── app.py           ← FastAPI-Server, API-Endpunkte, Hintergrundtasks
│   ├── main.py          ← Sensorauslesung (läuft auf dem Raspberry Pi)
│   ├── database.py       ← Datenbankverbindung (MariaDB/pymysql)
│   ├── requirements.txt  ← Python-Abhängigkeiten
│   ├──
│   ├── modules/
│   │   └── regressionsanalyse.py ← OLS-Regression + Personenschätzung
│   ├──
│   ├── routes/
│   │   ├── data.py       ← /api/data/* (Verlauf, Tabelle, Statistiken)
│   │   ├── estimator.py  ← /api/estimator/* (VOC-Baseline kalibrieren)
│   │   ├── occupancy.py  ← /api/occupancy/* (aktuelle Personenzahl)
│   │   └── regression.py ← /api/regression/* (Modell, Scatter, Vorhersage)
│   ├──
│   └── rpi_sensors/
│       ├── bme680_sensor.py ← BME680: Temperatur, Feuchte, Druck, VOC
│       ├── motion_sensor.py ← RCWL-0516: Bewegungserkennung
│       ├── sensors.py      ← Kombinierte Sensorauslesung
│       └── config.py       ← GPIO-Pin-Konfiguration
├──
└── frontend/
    ├── index.html        ← Einseitige Webanwendung (SPA)
    ├── css/
    │   └── style.css     ← Dunkles Design, Layout
    ├── js/
    │   └── dashboard.js  ← Tab-Navigation, API-Aufrufe, Diagramme

```

2. Entwicklungsdokumentation

08. Oktober 2025

Erstmalige Sensoranbindung & SSH-Konfiguration

In dieser Sitzung wurde der **BME680-Sensor** erstmalig an den Raspberry Pi angeschlossen und in Betrieb genommen. Die GPIO-Pinbelegung wurde festgelegt und die I²C-Kommunikation zwischen Sensor und Pi konfiguriert. Ein erster Testskript las die Rohwerte (Temperatur, Luftfeuchtigkeit, Luftdruck, Gaswiderstand) aus und gab sie in der Konsole aus.

Problem – SSH-Konfiguration: Zu Beginn bestanden Schwierigkeiten beim SSH-Zugriff auf den Raspberry Pi. Die SSH-Konfiguration musste angepasst werden, um einen stabilen Fernzugriff über **Visual Studio Code** (Remote-SSH-Extension) zu ermöglichen. Nach erfolgreicher Konfiguration konnte der Pi direkt aus VSCode heraus entwickelt und getestet werden.

- SSH-Zugriff konfiguriert und VSCode Remote-Verbindung eingerichtet
- I²C-Schnittstelle auf dem Raspberry Pi aktiviert
- BME680-Bibliothek installiert und getestet
- Erste erfolgreiche Sensorauslesung bestätigt

29. Oktober 2025

Anbindung und erstmaliges Coden

Aufbauend auf den ersten Tests wurde ein strukturiertes Python-Skript entwickelt, das die Sensorwerte regelmäßig ausliest und für eine spätere Speicherung vorbereitet. Die Grundstruktur des Projekts (Ordnerstruktur, Modulaufteilung) wurde festgelegt.

- Sensorauslesefunktion in ein eigenes Modul ausgelagert
- Erste Implementierung einer Messschleife
- Projektstruktur angelegt

05. November 2025

Neuer Sensor, Bewegungserkennung & defekter BME680

Der **RCWL-0516** Mikrowellen-Bewegungssensor wurde ergänzt. Das Ausleseskript wurde erweitert, sodass beide Sensoren gleichzeitig abgefragt werden. Der Bewegungssensor liefert ein binäres Signal (Bewegung erkannt / nicht erkannt), das als Plausibilitätsprüfung für die spätere Personenschätzung dient.

Problem – Defekter BME680: In dieser Phase lieferte der vorhandene BME680-Sensor keine verwertbaren Temperaturwerte mehr. Nach Diagnose wurde der Sensor als defekt eingestuft. Das Problem wurde durch den Austausch gegen ein neues BME680-Modul behoben.

- RCWL-0516 an GPIO-Pin angebunden
- Parallele Auslesung beider Sensoren implementiert
- Defekter BME680 identifiziert und durch Ersatzsensor ausgetauscht
- Bewegungsstatus wird gemeinsam mit Klimadaten erfasst

12. November 2025

Erstmalige Zielerfüllung – Datenbankbindung & 300s-Messintervall

Das System wurde auf ein Messintervall von **300 Sekunden** (5 Minuten) konfiguriert. Die **MariaDB-Datenbank** wurde auf dem Raspberry Pi eingerichtet und die Datenbankbindung über pymysql implementiert. Ab diesem Zeitpunkt werden alle Messwerte persistent gespeichert.

- MariaDB-Datenbank sensor_db und Tabelle sensor_data erstellt
- 300s-Messintervall konfiguriert
- Daten werden dauerhaft gespeichert – erste Datenreihen gesammelt
- Bewegungs- und Klimadaten werden gemeinsam in die DB geschrieben

04. Februar 2026

Website-Aufbau & Backend-Entwicklung

Zu diesem Zeitpunkt wurde die Webentwicklung gestartet. Das Backend wurde als **FastAPI**-Anwendung aufgebaut. Die erste `index.html` und die grundlegende API-Struktur entstanden.

Problem – Python Virtual Environment: Zunächst wurde eine Python Virtual Environment (venv) eingerichtet, um die Abhängigkeiten zu isolieren. Dies führte jedoch zu Problemen bei der Ausführung auf dem Raspberry Pi (Bibliothekskonflikt mit systemweiten GPIO-Paketen). Die Virtual Environment wurde daher wieder entfernt und alle Pakete systemweit installiert.

- FastAPI-Server (`app.py`) eingerichtet mit Uvicorn
- Virtual Environment (venv) zunächst erstellt, dann wegen Kompatibilitätsproblemen entfernt
- Abhängigkeiten systemweit auf dem Pi installiert
- Erste HTML-Oberfläche mit Platzhaltern erstellt
- REST-API-Endpunkte für Sensordaten definiert (`/api/data/latest`, `/api/data/history`)
- CORS-Middleware aktiviert für Browser-Zugriff

11. Februar 2026

Website & Backend-Weiterentwicklung

Das Frontend wurde zur vollständigen Single-Page-Application ausgebaut. Chart.js wurde als Diagramm-Bibliothek integriert. Die Tabs Dashboard, Sensoren und Verlauf wurden implementiert. Im Backend entstanden zusätzliche API-Endpunkte für Statistiken und die Datentabelle.

- Chart.js integriert – Temperatur-, Luftfeuchtigkeits- und Personenverlauf-Diagramme
- Tab-Navigation (Dashboard, Regression, Sensoren, Verlauf)
- Tabellen-Endpunkt mit Paginierung implementiert
- Automatische Aktualisierung alle 30 Sekunden

18. Februar 2026

Anpassung an Zielkriterien – Regressionsanalyse & Extension Board

Die **lineare Regressionsanalyse** wurde vollständig implementiert. Das Modell wird mit den Daten der letzten 48 Stunden trainiert und berechnet den Zusammenhang zwischen Personenanzahl und Raumtemperatur. Vorhersagen für die Szenarien „Leer“, „Halb“ und „Voll“ (0, 60, 120 Personen) werden angezeigt. Der Scatterplot visualisiert Messpunkte und Regressionsgerade.

Problem – Extension Board defekt: Das bisher verwendete GPIO-Extension Board konnte plötzlich keine Signale mehr korrekt weiterleiten. Die Sensoren lieferten keine verwertbaren Werte mehr. Nach Diagnose wurde das Extension Board als Fehlerquelle identifiziert. **Lösung:** Die Sensorkabel (BME680 und RCWL-0516) wurden direkt an die GPIO-Pins des Raspberry Pi angeschlossen und das Extension Board vollständig entfernt. Damit funktionierte das System wieder stabil (siehe Schaltungsfotos).

- **Extension Board entfernt** – direkte Verkabelung an Raspberry Pi GPIO-Pins
- OLS-Regression implementiert: Berechnung von Steigung (a), Achsenabschnitt (b) und R^2 -Score
- Automatisches Modell-Training beim Serverstart (nach 10s) und stündlich
- Scatterplot: Personen (x) vs. Temperatur (y) mit Regressionsgerade
- Szenarien-Tabelle: Vorhersage für 0, 60, 120 Personen

- Datenbeschränkung auf 30 Tage implementiert (tägliche Bereinigung)

25. Februar 2026

Finale Anpassungen & Deployment

Letzte Verbesserungen: URL-Routing (jeder Tab erhält eine eigene URL), manueller Aktualisierungs-Button für das Regressionsmodell, stündliches Neutraining des Modells mit den letzten 48h Daten.

- URL-Routing: /, /regression, /sensors, /history
- „Jetzt trainieren“-Button im Regression-Tab
- Trainingsintervall auf 1 Stunde verkürzt (Datenfenster bleibt 48h)
- Deployment und Test auf dem Raspberry Pi

3. Schaltung

Die Schaltung besteht aus einem **Raspberry Pi 4 Model B** als Zentraleinheit, dem **BME680**-Umgebungssensor (über I²C angebunden) sowie dem **RCWL-0516**-Bewegungssensor (digitaler GPIO-Eingang).

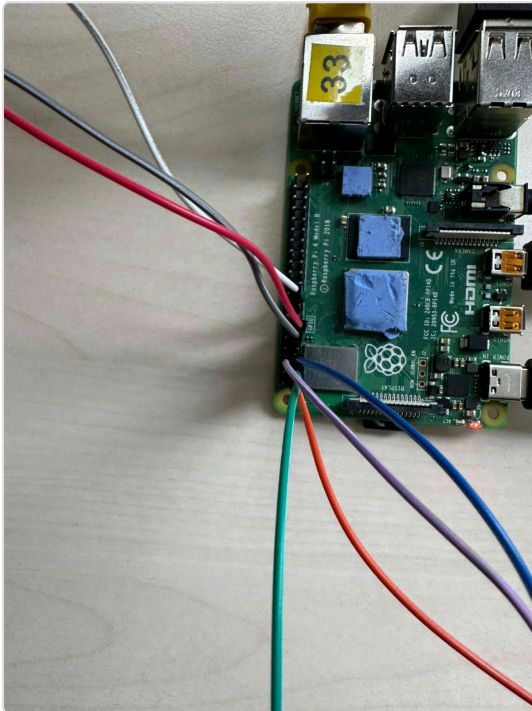


Abb. 1 – Raspberry Pi mit angeschlossenen Sensorkabeln

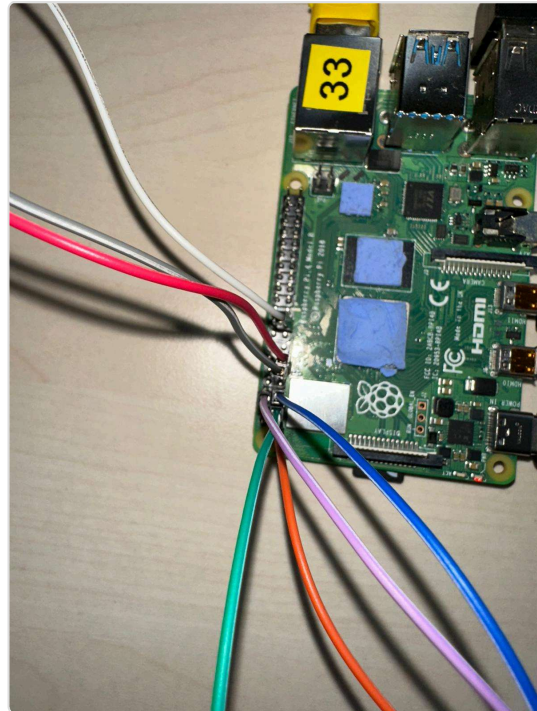


Abb. 2 – Kabelverbindungen GPIO-Pins

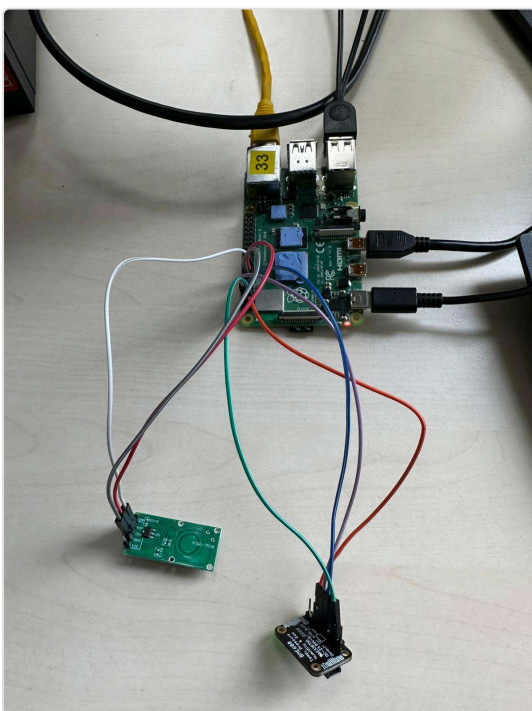


Abb. 3 – BME680 Sensor-Anschluss

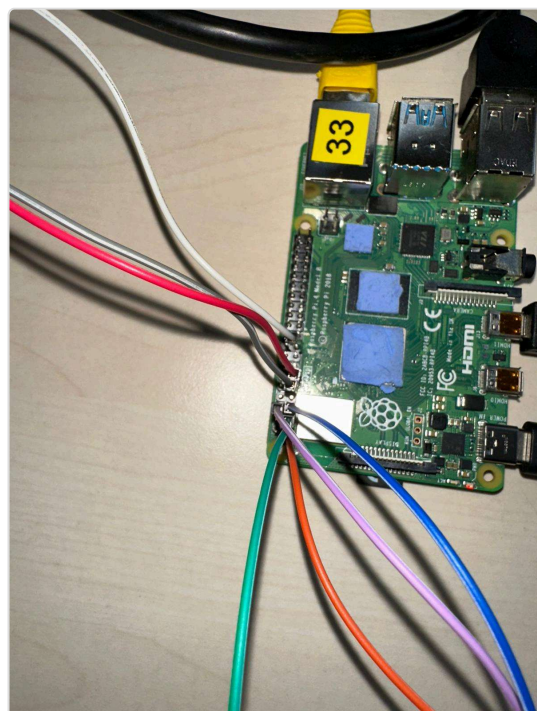


Abb. 4 – RCWL-0516 Bewegungssensor

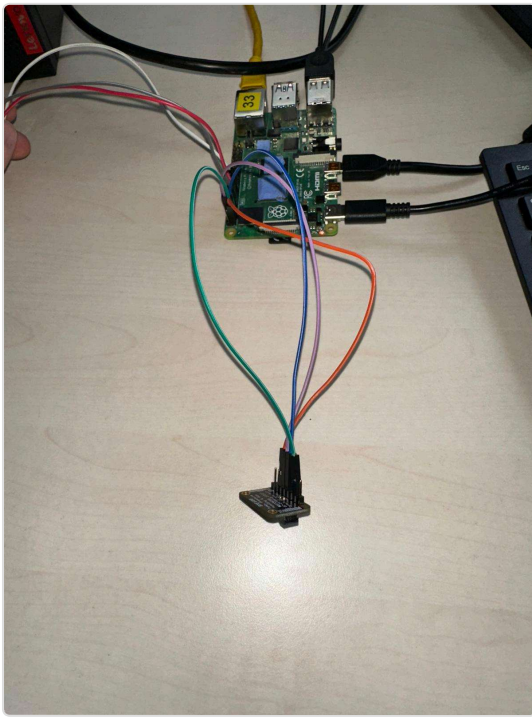


Abb. 5 – Gesamtaufbau Schaltung



Abb. 6 – Sensorplatine Detailansicht

4. GUI – Weboberfläche

Die Weboberfläche wird über den FastAPI-Server auf Port 8000 des Raspberry Pi bereitgestellt und ist im lokalen Netzwerk über die IP-Adresse des Pi erreichbar. Die Seite gliedert sich in vier Tabs: **Dashboard**, **Regression**, **Sensoren** und **Verlauf**.

4.1 Dashboard

Das Dashboard zeigt die aktuellen Messwerte (Temperatur, Luftfeuchtigkeit, VOC-Gaswiderstand, geschätzte Personenanzahl) sowie drei Verlaufsdiagramme für die letzten 24 Stunden.

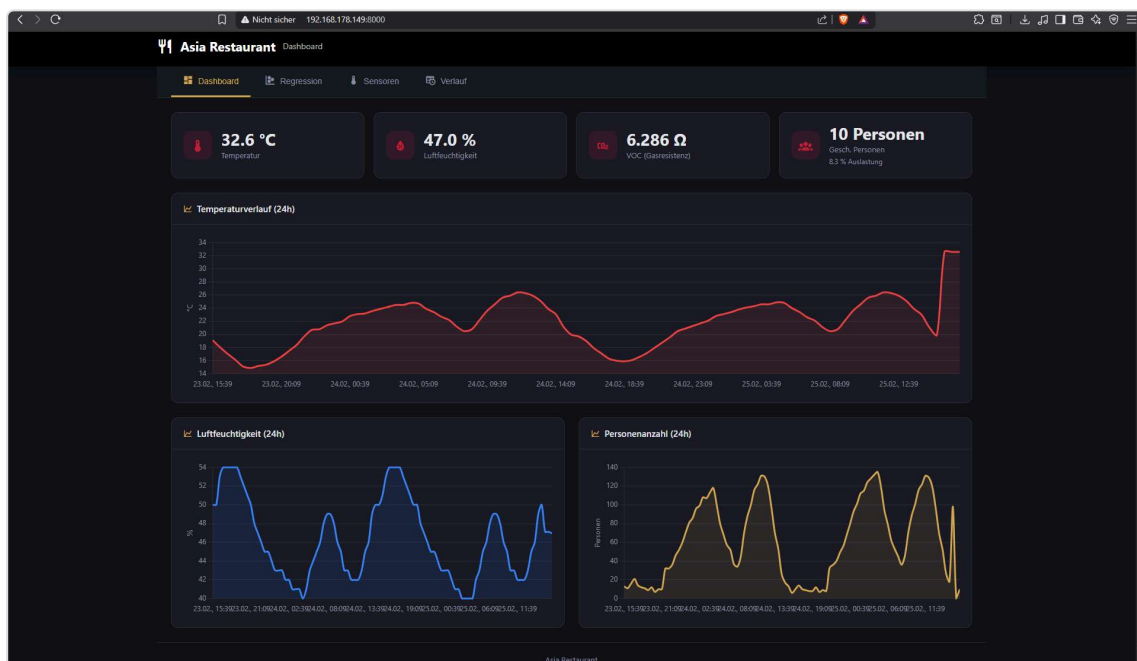


Abb. 7 – Dashboard: Aktuelle Werte und Verlaufsdiagramme (Temperatur, Luftfeuchtigkeit, Personenanzahl)

4.2 Regressionsanalyse

Der Regression-Tab zeigt das trainierte lineare Regressionsmodell mit Steigung (a), Achsenabschnitt (b), R^2 -Score und Anzahl der Datenpunkte. Im Scatterplot sind alle Messpunkte sowie die Regressionsgerade dargestellt. Die Szenarien-Tabelle gibt Temperaturvorhersagen für 0, 60 und 120 Personen aus.

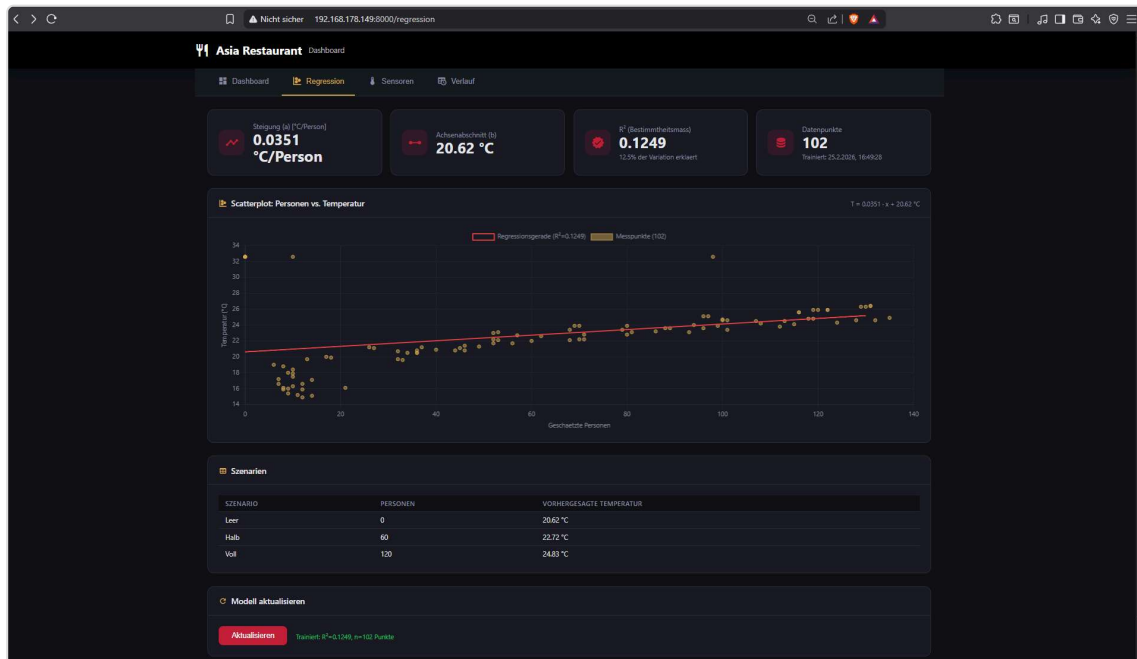


Abb. 8 – Regression: Scatterplot (Personen vs. Temperatur), Regressionsformel $T = 0.0351 \cdot x + 20.62 \text{ °C}$, $R^2 = 0.1249$, Szenarien-Tabelle und „Aktualisieren“-Button

4.3 Sensoren-Tab

Der Sensoren-Tab zeigt aktuelle Einzelwerte aller Sensoren, ermöglicht die VOC-Baseline-Kalibrierung (für leerem Raum) und zeigt historische Diagramme für Temperatur, VOC/Gaswiderstand und Luftdruck.

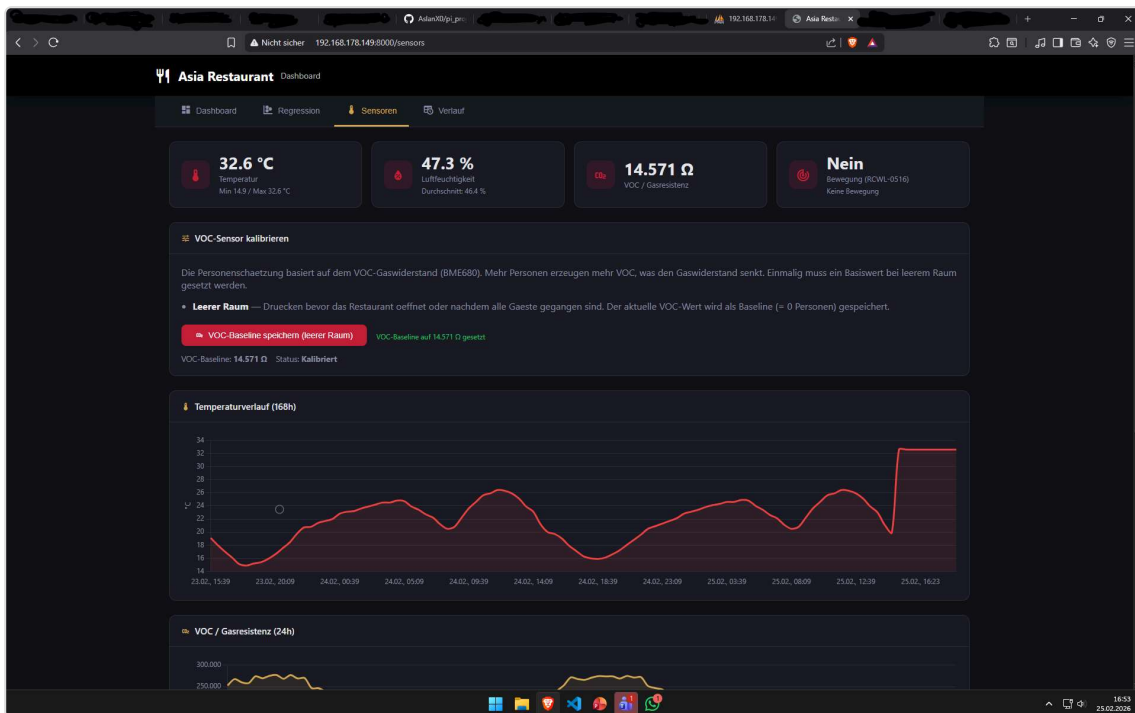
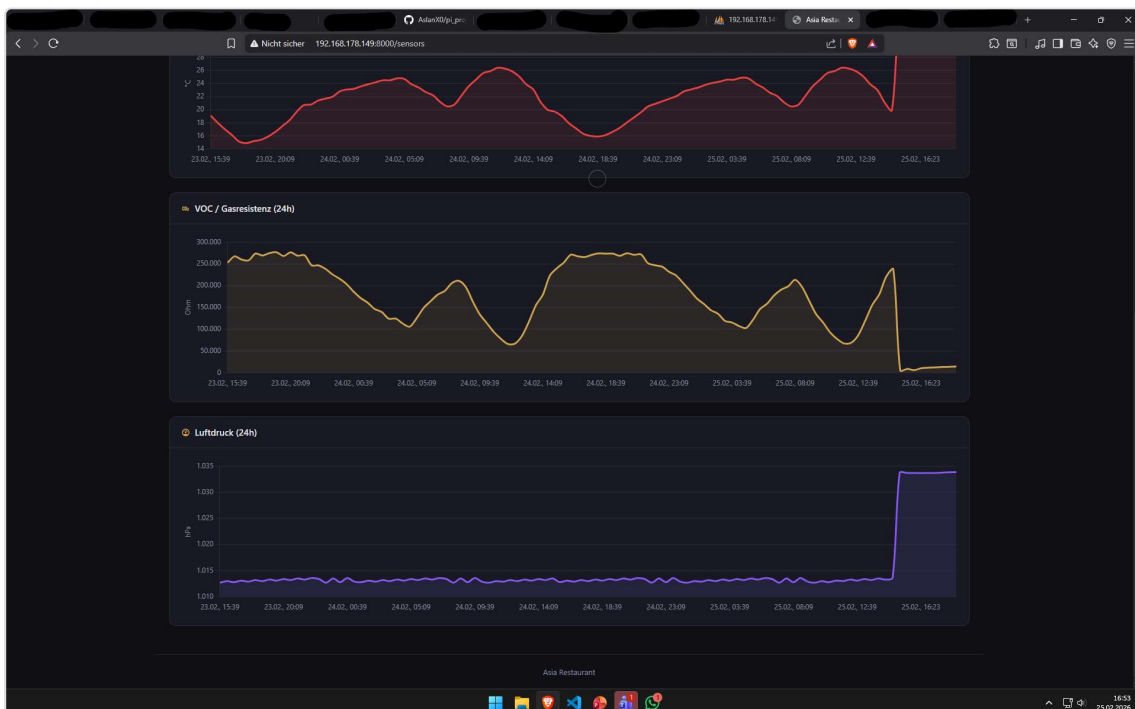
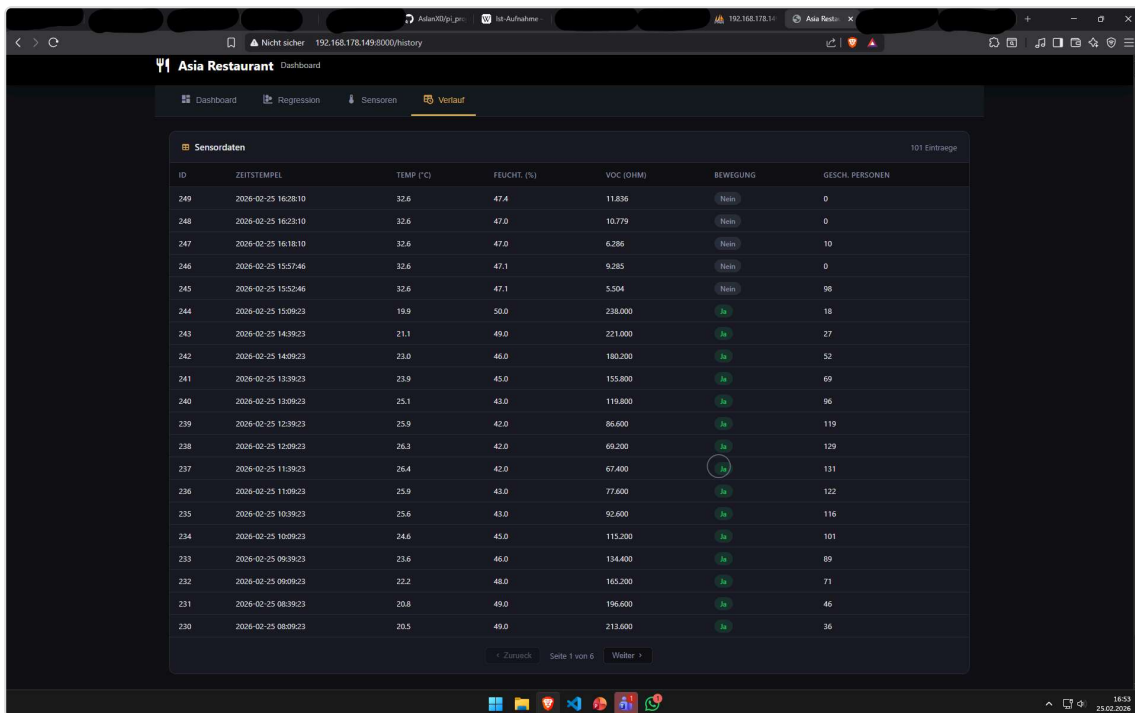


Abb. 9 – Sensoren: Aktuelle Werte, VOC-Kalibrierung und Temperaturverlauf (168h)



4.4 Verlauf-Tab

Der Verlauf-Tab zeigt alle gespeicherten Sensordaten in einer paginierten Tabelle (ID, Zeitstempel, Temperatur, Luftfeuchtigkeit, VOC, Bewegung, geschätzte Personen). Die Daten werden für maximal 30 Tage aufbewahrt.



ID	ZEITSTEMPEL	TEMP (°C)	FEUCHT. (%)	VOC (OHM)	BEWEGUNG	GESCH. PERSONEN
249	2026-02-25 16:28:10	32,5	47,4	11.836	Nein	0
248	2026-02-25 16:23:10	32,6	47,0	10.779	Nein	0
247	2026-02-25 16:18:10	32,6	47,0	6.286	Nein	10
246	2026-02-25 15:57:46	32,6	47,1	9.285	Nein	0
245	2026-02-25 15:52:46	32,6	47,1	5.504	Nein	98
244	2026-02-25 15:09:23	19,9	50,0	238.000	Ja	18
243	2026-02-25 14:39:23	21,1	49,0	221.000	Ja	27
242	2026-02-25 14:09:23	23,0	46,0	180.200	Ja	52
241	2026-02-25 13:39:23	23,9	45,0	155.800	Ja	69
240	2026-02-25 13:09:23	25,1	43,0	119.800	Ja	96
239	2026-02-25 12:39:23	25,9	42,0	86.600	Ja	119
238	2026-02-25 12:09:23	26,3	42,0	69.200	Ja	129
237	2026-02-25 11:39:23	26,4	42,0	67.400	Ja	131
236	2026-02-25 11:09:23	25,9	43,0	77.600	Ja	122
235	2026-02-25 10:39:23	25,6	43,0	92.600	Ja	116
234	2026-02-25 10:09:23	24,6	45,0	115.200	Ja	101
233	2026-02-25 09:39:23	23,6	46,0	134.400	Ja	89
232	2026-02-25 09:09:23	22,2	48,0	165.200	Ja	71
231	2026-02-25 08:39:23	20,8	49,0	196.600	Ja	46
230	2026-02-25 08:09:23	20,5	49,0	213.600	Ja	36

Abb. 11 – Verlauf: Paginierte Sensordatentabelle mit 101 Einträgen

4.5 Sensor-Erfassung (Terminal)

Das Sensor-Skript (main.py) läuft parallel zum Webserver und liest alle 300 Sekunden die Sensoren aus. Die Daten werden direkt in die Datenbank geschrieben.

```
it@raspberrypi:~/project/backend $ python main.py
Nächste Messung in 300s
[2026-02-25 16:23:10] Lese Sensoren...
Daten gespeichert
Nächste Messung in 300s
[2026-02-25 16:28:10] Lese Sensoren...
Daten gespeichert
Nächste Messung in 300s
[2026-02-25 16:33:10] Lese Sensoren...
Daten gespeichert
Nächste Messung in 300s
```

Abb. 12 – Terminal: python main.py – Sensorauslesung alle 300s, Daten werden automatisch gespeichert

5. Fazit

Das Projekt wurde erfolgreich umgesetzt. Alle im Lastenheft definierten Anforderungen sind erfüllt:

Anforderung	Status
Sensorauslesung (BME680 + RCWL-0516) alle 300s	Erfüllt
Persistente Datenbankanbindung (MariaDB)	Erfüllt
Weboberfläche mit Echtzeit-Anzeige	Erfüllt
Regressionsmodell mit Datenfenster 48h	Erfüllt
Berechnung von Steigung (a), Achsenabschnitt (b), R^2	Erfüllt
Scatterplot mit Regressionsgerade	Erfüllt
Vorhersagen für 0 / 60 / 120 Personen	Erfüllt
Datenbeschränkung auf 30 Tage	Erfüllt
URL-Routing für alle Tabs	Erfüllt

Herausforderungen

Die größten technischen Herausforderungen lagen in der stabilen Netzwerkkonfiguration des Raspberry Pi, der korrekten I²C-Kommunikation mit dem BME680-Sensor sowie der Implementierung des Regressionsmodells ohne externe ML-Bibliotheken (nur NumPy). Auch die Entwicklung einer vollständigen SPA-Webanwendung mit History-API-Routing stellte eine Herausforderung dar.